

Experiment 'p_4_2_c_cumulative_totals_seq' Results

June 3, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_4_2_c_cumulative_totals_seq

Natural language statement: Write a method that takes a sequence of integers and returns cumulative totals. For example, if the input is [1, 7, 2, 9], the method should return [1, 8, 10, 19].

Method signature: p_4_2_c_cumulative_totals_seq(inputs: seq<int>) returns (totals: seq<int>)

Ensures

- $|totals| == |inputs|$
- forall $i :: 0 \leq i < |inputs| \implies totals[i] == (if\ i == 0\ then\ inputs[0]\ else\ totals[i - 1] + inputs[i])$

Requires

- $0 < |inputs|$

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that takes a sequence of integers and returns cumulative totals. For
→ example, if the input is [1, 7, 2, 9], the method should return [1, 8, 10, 19].

The signature should be:

p_4_2_c_cumulative_totals_seq(inputs: seq<int>) returns (totals: seq<int>)

The method should respect the following contract:

requires 0 < |inputs|, ensures |totals| == |inputs|, ensures forall i :: 0 <= i < |inputs|
→ | ==> totals[i] == (if i == 0 then inputs[0] else totals[i - 1] + inputs[i])

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dme7Q8IQJwe3DgJ2dE3oKxfczQWom

Model: gpt-5.5-2026-04-23

Created at: 1780486332

Finish reason: stop

Usage: 438 tokens in, and 5178 tokens out

```
{
  totals := [inputs[0]];
  assert |totals| == 1;
  assert totals[0] == inputs[0];
  assert forall j :: 0 <= j < 1 ==> totals[j] == (if j == 0 then inputs[0] else totals[j
    → - 1] + inputs[j]) by {
    forall j | 0 <= j < 1
      ensures totals[j] == (if j == 0 then inputs[0] else totals[j - 1] + inputs[j])
      {
        assert j == 0;
        assert totals[j] == inputs[0];
      }
  }

  var i := 1;
  while i < |inputs|
    invariant 1 <= i <= |inputs|
    invariant |totals| == i
```

```

invariant forall j :: 0 <= j < i ==> totals[j] == (if j == 0 then inputs[0] else
  ↪ totals[j - 1] + inputs[j])
{
  var oldTotals := totals;
  assert |oldTotals| == i;

  totals := oldTotals + [oldTotals[i - 1] + inputs[i]];
  assert |totals| == i + 1;

  assert forall j :: 0 <= j < i + 1 ==> totals[j] == (if j == 0 then inputs[0] else
    ↪ totals[j - 1] + inputs[j]) by {
    forall j | 0 <= j < i + 1
      ensures totals[j] == (if j == 0 then inputs[0] else totals[j - 1] + inputs[j])
    {
      if j < i {
        assert 0 <= j < |oldTotals|;
        assert totals[j] == oldTotals[j];
        assert oldTotals[j] == (if j == 0 then inputs[0] else oldTotals[j - 1] + inputs[
          ↪ j]);
        if j == 0 {
          assert totals[j] == inputs[0];
        } else {
          assert 0 <= j - 1 < |oldTotals|;
          assert totals[j - 1] == oldTotals[j - 1];
          assert totals[j] == totals[j - 1] + inputs[j];
        }
      } else {
        assert j == i;
        assert j != 0;
        assert totals[j] == oldTotals[i - 1] + inputs[i];
        assert 0 <= i - 1 < |oldTotals|;
        assert totals[i - 1] == oldTotals[i - 1];
        assert j - 1 == i - 1;
        assert totals[j - 1] == oldTotals[i - 1];
        assert inputs[j] == inputs[i];
        assert totals[j] == totals[j - 1] + inputs[j];
      }
    }
  }

  i := i + 1;
}

assert i == |inputs|;
assert |totals| == |inputs|;
assert forall j :: 0 <= j < |inputs| ==> totals[j] == (if j == 0 then inputs[0] else
  ↪ totals[j - 1] + inputs[j]) by {
  forall j | 0 <= j < |inputs|
    ensures totals[j] == (if j == 0 then inputs[0] else totals[j - 1] + inputs[j])
  {
    assert j < i;

```

```

    }
  }
}

```

Final Program

```

method p_4_2_c_cumulative_totals_seq(inputs: seq<int>) returns (totals: seq<int>)
  requires 0 < |inputs|
  ensures |totals| == |inputs|
  ensures forall i :: 0 <= i < |inputs| ==> totals[i] == (if i == 0 then inputs[0]
    ↪ else totals[i - 1] + inputs[i])
{
  totals := [inputs[0]];
  assert |totals| == 1;
  assert totals[0] == inputs[0];
  assert forall j :: 0 <= j < 1 ==> totals[j] == (if j == 0 then inputs[0] else totals[j
    ↪ - 1] + inputs[j]) by {
    forall j | 0 <= j < 1
      ensures totals[j] == (if j == 0 then inputs[0] else totals[j - 1] + inputs[j])
    {
      assert j == 0;
      assert totals[j] == inputs[0];
    }
  }
}

var i := 1;
while i < |inputs|
  invariant 1 <= i <= |inputs|
  invariant |totals| == i
  invariant forall j :: 0 <= j < i ==> totals[j] == (if j == 0 then inputs[0] else
    ↪ totals[j - 1] + inputs[j])
{
  var oldTotals := totals;
  assert |oldTotals| == i;

  totals := oldTotals + [oldTotals[i - 1] + inputs[i]];
  assert |totals| == i + 1;

  assert forall j :: 0 <= j < i + 1 ==> totals[j] == (if j == 0 then inputs[0] else
    ↪ totals[j - 1] + inputs[j]) by {
    forall j | 0 <= j < i + 1
      ensures totals[j] == (if j == 0 then inputs[0] else totals[j - 1] + inputs[j])
    {
      if j < i {
        assert 0 <= j < |oldTotals|;
        assert totals[j] == oldTotals[j];
        assert oldTotals[j] == (if j == 0 then inputs[0] else oldTotals[j - 1] + inputs[
          ↪ j]);
      }
      if j == 0 {
        assert totals[j] == inputs[0];
      }
    }
  }
}

```

```

    } else {
      assert 0 <= j - 1 < |oldTotals|;
      assert totals[j - 1] == oldTotals[j - 1];
      assert totals[j] == totals[j - 1] + inputs[j];
    }
  } else {
    assert j == i;
    assert j != 0;
    assert totals[j] == oldTotals[i - 1] + inputs[i];
    assert 0 <= i - 1 < |oldTotals|;
    assert totals[i - 1] == oldTotals[i - 1];
    assert j - 1 == i - 1;
    assert totals[j - 1] == oldTotals[i - 1];
    assert inputs[j] == inputs[i];
    assert totals[j] == totals[j - 1] + inputs[j];
  }
}
}

i := i + 1;
}

assert i == |inputs|;
assert |totals| == |inputs|;
assert forall j :: 0 <= j < |inputs| ==> totals[j] == (if j == 0 then inputs[0] else
  ↪ totals[j - 1] + inputs[j]) by {
  forall j | 0 <= j < |inputs|
    ensures totals[j] == (if j == 0 then inputs[0] else totals[j - 1] + inputs[j])
  {
    assert j < i;
  }
}
}
}

```

Total Token Usage

Input tokens: 438

Output tokens: 5178

Reasoning tokens: 4316

Sum of ‘total tokens’: 5616

Experiment Timings

Overall Experiment started at 1780486332187, ended at 1780486424364, lasting 92177ms (92.18 seconds)

Iteration #1 started at 1780486332188, ended at 1780486424364, lasting 92176ms (92.18 seconds)